

Quantile GAM Regression

GAM.jl Contributors

Table of contents

Introduction	1
Setup	2
Simulate heteroscedastic data	2
Fit a single quantile GAM	2
Median regression ($\tau = 0.5$)	3
Compare with mean regression	4
Fit multiple quantiles	4
Individual quantile fits	4
Using <code>mqgam</code> for simultaneous fitting	5
Quantile crossing check	5
Verify quantile coverage	6
The ELF family	6
Summary	7

Introduction

Standard GAMs model the **conditional mean** $E[Y \mid X]$. **Quantile regression** instead models conditional quantiles $Q_\tau(Y \mid X)$, providing a more complete picture of the response distribution—especially useful when:

- The variance changes with covariates (heteroscedasticity)
- Interest lies in tail behavior (e.g., 90th percentile)
- The distribution is skewed

GAM.jl implements quantile GAMs following the **qgam** approach (Fasiolo et al., 2021), using the **Extended Log-F (ELF)** loss as a smooth approximation to the pinball (check) loss:

$$\rho_\tau(u) = u(\tau - \mathbb{1}_{u < 0})$$

The ELF family provides a smooth, twice-differentiable loss that converges to the pinball loss, enabling standard penalized likelihood fitting with automatic smoothing parameter selection.

Setup

```
using GAM
using StatsAPI: predict, fitted, residuals
using DataFrames
using Random
using Statistics
```

Simulate heteroscedastic data

We create data where the conditional variance increases with x :

$$y = \sin(2\pi x) + (0.2 + 0.8x)\varepsilon, \quad \varepsilon \sim N(0, 1)$$

```
Random.seed!(42)
n = 500
x = rand(n)

noise = randn(n)
y = sin.(2 .* x) .+ (0.2 .+ 0.8 .* x) .* noise

df = DataFrame(y=y, x=x)
println("Data: n = $n")
println("y range: [$(round(minimum(y), digits=2)), $(round(maximum(y), digits=2))]")
println("Mean y: $(round(mean(y), digits=3))")
```

```
Data: n = 500
y range: [-3.4, 2.01]
Mean y: -0.012
```

Fit a single quantile GAM

The `qgam` function fits a GAM for a single quantile τ . Internally it:

1. Fits a preliminary Gaussian GAM to estimate variance

2. Computes the smoothness constant
3. Tunes the learning rate via grid search
4. Fits the final model with the ELF family

Median regression ($\tau = 0.5$)

```
m_50 = qgam(@gam_formula(y ~ s(x, k=20, bs=:cr)), df, 0.5)
```

Generalized Additive Model

Formula: $y \sim 1$

Family: ELF

Link: IdentityLink

Method: REML

Parametric coefficients:

	Coef.	Std. Error	z	Pr(> z)
(Intercept)	-0.011804	0.0103526	-1.14	0.2542

Approximate significance of smooth terms:

Smooth	edf	Ref.df
s(x,bs=cr)	16.77	19

R^2 (adj) = 0.497 Deviance explained = 51.4%
n = 500

```
yhat_50 = predict(m_50)
println("Median regression fitted range: [$(round(minimum(yhat_50), digits=3)), $(round(maximum(yhat_50), digits=3))]
```

Median regression fitted range: [-1.173, 1.001]

Compare with mean regression

```
m_mean = gam(@gam_formula(y ~ s(x, k=20, bs=:cr)), df)
yhat_mean = predict(m_mean)

println("Mean regression fitted range: [$(round(minimum(yhat_mean), digits=3)), $(round(maximum(yhat_mean), digits=3))]", digits=3)
println("Correlation (mean vs median): $(round(cor(yhat_mean, yhat_50), digits=4))")
```

Mean regression fitted range: [-0.977, 0.976]
Correlation (mean vs median): 0.9952

For symmetric errors, mean and median regression give similar results. With heteroscedastic or skewed data, they can diverge.

Fit multiple quantiles

Individual quantile fits

```
quantiles = [0.1, 0.25, 0.5, 0.75, 0.9]

fits = Dict{Float64, Any}()
for qu in quantiles
    fits[qu] = qgam(@gam_formula(y ~ s(x, k=20, bs=:cr)), df, qu)
end
```

Examine fitted ranges for each quantile:

```
ord = sortperm(x)
for qu in quantiles
    yhat = predict(fits[qu])
    println(" = $qu: fitted range [$(round(minimum(yhat), digits=3)), $(round(maximum(yhat), digits=3))]", digits=3)
end
```

= 0.1: fitted range [-1.334, -1.334]
= 0.25: fitted range [-0.706, 0.225]
= 0.5: fitted range [-1.173, 1.001]
= 0.75: fitted range [-0.055, 0.765]
= 0.9: fitted range [1.087, 1.087]

Using `mqgam` for simultaneous fitting

`mqgam` fits all quantiles simultaneously, sharing the preliminary variance estimate for efficiency:

```
mq = mqgam(@gam_formula(y ~ s(x, k=20, bs=:cr)), df, quantiles)
```

```
(fits = Dict{Float64, Any}(0.5 => GamModel(n_smooth=1, edf=17.8, deviance=1531.98), 0.9 => G
```

```
println("Quantiles fitted: ", mq.quantiles)
println("Number of models: ", length(mq.fits))
```

```
Quantiles fitted: [0.1, 0.25, 0.5, 0.75, 0.9]
Number of models: 5
```

Access individual model fits:

```
for qu in quantiles
    yhat = predict(mq.fits[qu])
    println(" = $qu: fitted range [$(round(minimum(yhat), digits=3)), $(round(maximum(yhat),
end
```

```
 = 0.1: fitted range [-1.334, -1.334]
 = 0.25: fitted range [-0.706, 0.225]
 = 0.5: fitted range [-1.173, 1.001]
 = 0.75: fitted range [-0.055, 0.765]
 = 0.9: fitted range [1.087, 1.087]
```

Quantile crossing check

A known issue with independent quantile fits is **quantile crossing**, where estimated quantile curves cross each other. Let's check:

```
x_sorted = x[ord]
predictions = Dict{qu => predict(mq.fits[qu])[ord] for qu in quantiles)

n_crossings = 0
for i in 1:n
    vals = [predictions[qu][i] for qu in quantiles]
    if !issorted(vals)
```

```

        n_crossings += 1
    end
end
println("Quantile crossings: $n_crossings out of $n observations ($(round(100*n_crossings/n,

```

Quantile crossings: 273 out of 500 observations (54.6%)

Verify quantile coverage

For well-calibrated quantile estimates, approximately $\tau \times 100\%$ of observations should fall below the τ -th quantile curve:

```

for qu in quantiles
    yhat = predict(mq.fits[qu])
    coverage = mean(y .< yhat)
    println(" = $qu: empirical coverage = $(round(coverage, digits=3)) (target = $qu)")
end

```

```

= 0.1: empirical coverage = 0.1 (target = 0.1)
= 0.25: empirical coverage = 0.318 (target = 0.25)
= 0.5: empirical coverage = 0.518 (target = 0.5)
= 0.75: empirical coverage = 0.662 (target = 0.75)
= 0.9: empirical coverage = 0.9 (target = 0.9)

```

The ELF family

The Extended Log-F family used internally can be accessed directly:

```

elf = ELFFamily(qu=0.5, co=0.1)
println("ELF family for median: quantile = ", elf.qu)

```

ELF family for median: quantile = 0.5

The pinball loss for a given quantile τ :

```

pl = pinball_loss(y, predict(mq.fits[0.5]), 0.5)
println("Pinball loss (=0.5): ", round(pl, digits=3))

for qu in quantiles
    pl = pinball_loss(y, predict(mq.fits[qu]), qu)
    println("Pinball loss (=$qu): $(round(pl, digits=3))")
end

```

```

Pinball loss (=0.5): 118.123
Pinball loss (=0.1): 91.852
Pinball loss (=0.25): 126.573
Pinball loss (=0.5): 118.123
Pinball loss (=0.75): 110.594
Pinball loss (=0.9): 67.791

```

Summary

Feature	GAM.jl	R qgam
Single quantile	<code>qgam(formula, data,)</code>	<code>qgam(formula, data=dat, qu=)</code>
Multiple quantiles	<code>mqgam(formula, data, [, , ...])</code>	<code>mqgam(formula, data=dat, qu=c(...))</code>
Loss function	ELF (Extended Log-F)	ELF
Smoothing	Automatic (REML-based)	Automatic
Pinball loss	<code>pinball_loss(r,)</code>	<code>pinLoss(...)</code>
Formula syntax	<code>@gam_formula(y ~ s(x, k=20))</code>	<code>y ~ s(x, k=20)</code>

Quantile GAMs extend the GAM framework to model the full conditional distribution, making them especially valuable for heteroscedastic data where the spread changes with covariates.